

```
//CSE  
110;
```

Introduction

with Thomas Powell

Today

- Intros and Overview of the Course
 - Canvas, Slack, etc.
 - Logistics
 - Class Assumptions and Aims
 - Interactive Portion for today's topic
 - This season on 110 Amusements section
-

Who Are We?

Meet the Prof

Lecturer UCSD CSE Department



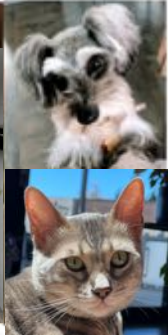
Industry CEO, Founder



Author



Some of my interests



Instructional Staff - TAs

- Rashi Bhanli
 - Devanshi Chadha
 - Sammed Kamate
 - Tony Li
 - Shelby Myrman
-
- The role of TAs in 110
-

Instructional Staff - Tutors

- Elaine Chao
 - Shravi Jain
 - Sofia Nguyen
-
- The role of tutors in 110
-

Class Specifics Time

Lectures, Labs, and Discussions, Tests

- **Lectures**

- You need to be here and we will try to do something interactive or something supportive of teams every time
- Attendance of course will be factored but given the reality of life allow up to 4 misses without harm
- No podcasting for many reasons I'll explain

- **Discussion**

- Not traditional and relates to teams

- **Labs**

- Flexibility in approach, support within time, some outside
- Drop 1
- Solo initial, changes as it goes to support pairing and teams

- **Testing**

- Midterms, Some topic quizzes, and a short interview

Preview of Our Project Efforts

- Process over Project Code
 - Work up front, but iterative and ...
 - Constant Effort
 - Small and steady for the win, or do as you do and be pained :-(
 - Big project made small to avoid space to hide
 - There is some flexing this quarter!
 - Relatively Fixed Tool Chain
 - To save you from yourself and provide the TAs the ability to help you!
 - Shortly, the premise (late week 2 unless something happens)
-

Team Preview

- Assigned
 - Size: ~8-10 may fluctuate for obvious reasons (drops! So we start at 10 or 11 always)
 - Balanced by upcoming survey responses = BE HONEST
 - Organization begins 1st week's weekend
-

Some Assumptions

- You know how to program at least somewhat decently
 - You have a reason you want to be here I hope beyond you must be...(REALISM STATEMENT FOLLOWS)
 - There is some expected outcome I hope, beyond the grade if I we both do our jobs right
 - Some of you will have worked (internship, part-time, etc.), but most likely, the majority will not
-

Warning No PA Zone Ahead



Agile Otter Tim @tottinge · Aug 27, 2020

I'm still surprised when dealing with new developers, how quick they are to drop into waterfall - ask for detailed requirements docs, design specs, etc.

I would get it if they were working in the 80s and 90s but when they have less than 10 years of experience?

62

42

181



Ron Jeffries @RonJeffries · Aug 28, 2020

I'm not surprised. It takes great experience to become comfortable that we can develop code so incrementally that we don't need to know in advance where we're going over the long haul.

And possibly there's a knock-on effect from school as well.

4

4

39



Thomas A. Powell @thomasapowell · Aug 28, 2020

In school students do expect ultra clear assignments & to drop into coding immediately, so inherently they are taught Waterfall. They also hate on it because told to, but want their view of its certainty—quite odd. I do my best to break this, but I am a few courses out of many.

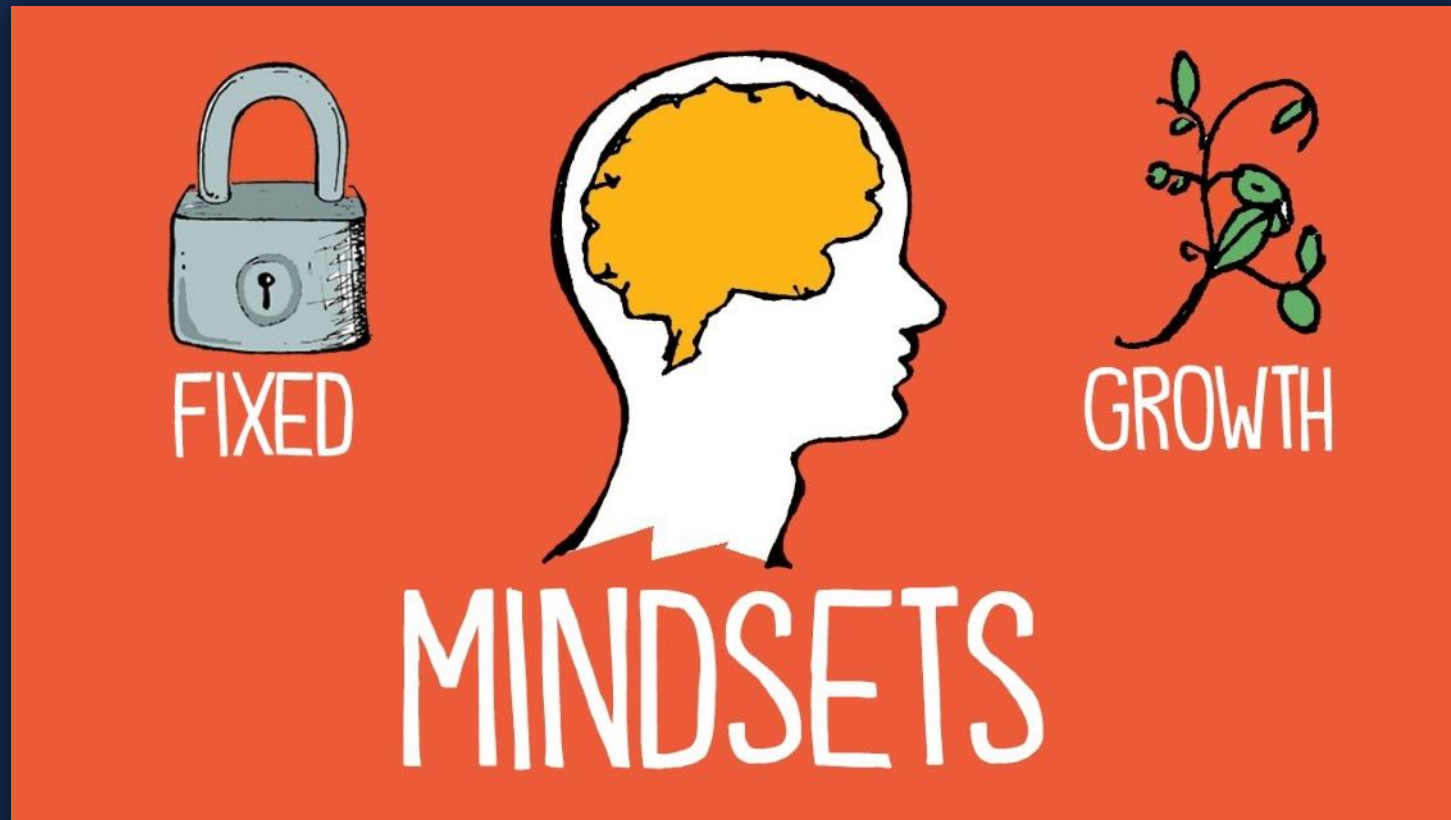


My Class Goal

To help students gain an initial and practical working knowledge of how to produce long-lived quality-focused software in an “engineered” fashion, balancing theoretical aspirations with the practicalities imposed by the development environment that will be encountered in the uncertain “real world.”

Given that this is **not possible in 10 weeks**, I also aim to instill the sense that this must be a continued deliberative and iterative approach to learning SE that moves the ownership of learning to where it belongs—to YOU.

...but where are we in our thinking?



Your Beliefs, My Beliefs, and OUR Beliefs

- You may not believe it, but many of you already have in some cases very strong beliefs about what software engineering is or should be
- We must get those out in the open. - Enter the upcoming Survey Assignment and a bit of this lecture
- If we can't define this and admit to our expectations, how will we even know if we did a good or bad job at it!?!

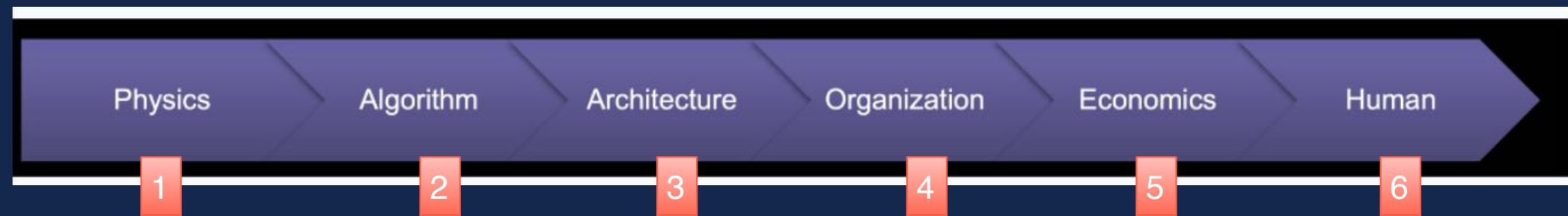
Topic

What is Software Engineering?

Activity Time

- Question 1 (How different)
 - Question 2 (Activities)
 - We will then discuss this so answer if you feel ok doing that for a positive signal capture.
 - Now modify your answers if you like
-

What's In Bounds for Us



An “Answer” Key?

Physics

Algorithm

Architecture

Organization

Economics

Human

Computer science

Software engineering

Do you think some things do not belong at all?

Nothing Spicy ... Just Observations

Software engineering is often treated as a branch of computer science. This is akin to regarding chemical engineering as a branch of chemistry. We need both chemists and chemical engineers, but they are different.

Modern Software Engineering

David Farley



One List of Software Engineering Activities

- Planning
 - Ideation, requirements gathering, domain exploration, etc.
- Design
 - Domain analysis/verification, system level, UI level, code level / architecture, exploratory dev and premise testing
- Implementation
 - Coding, Building (including the building of the system to do the work), Documentation
- Validation and Quality Assurance
 - Testing - unit, integration, user
 - Code Review
- Deployment
- Monitoring
 - Analytics - many types from broad to user to user, split testing, error tracking
- Maintenance
 - Bug fixing, refactoring

The last point suggests a lifecycle of software from birth to maturity and this means we should also consider the decommission step as well. This path is usually discussed as the **Software Development Life Cycle (SDLC)**

How About

- Meetings, meetings, meetings
 - Communicating in all forms (chat, email, issues, etc.)
 - Presenting work or ideas
 - Analyzing reports and data
 - Servicing various organizational requirements (OKRs, Personal Plans, Stack ranking peers, interviewing)
 - Learning new things to do the work you do or improve the work
 - This would include lots of reading
-

In Class Activity Part 2

- Fill out the percentage of time guess
 - Fill out the good and bad activities
 - Define software engineering
-

So What Is It?

Asking our Mighty Word “Friend”



Just...Engineering

- Engineering is derived from the Latin ingenium, meaning "cleverness" and ingeniare, meaning "to contrive, devise"

*“The **creative application of scientific principles to design or develop structures, machines, apparatus, or manufacturing processes, or works utilizing them singly or in combination; or to construct or operate the same with full cognizance of their design; or to forecast their behavior under specific operating conditions; all as respects an intended function, economics of operation or safety to life and property**”*

What is Software Engineering?



WIKIPEDIA
The Free Encyclopedia

[Main page](#)
[Contents](#)
[Featured content](#)

[Article](#) [Talk](#) [Read](#)

Software engineering

From Wikipedia, the free encyclopedia

Software engineering is the study and an application of [engineering](#) to the design, development and maintenance of software.^{[1][2][3]}

Why do we want SE?

- Predictability
 - Repeatability
 - Cost Effective
 - Higher Quality
-
- The alternative of course is a less predictable, not necessarily repeatable, variably cost and quality process and result.
-

Getting more precise...

Typical formal definitions of **software engineering** are:

- "the systematic application of scientific and technological knowledge, methods, and experience to the design, implementation, testing, and documentation of software";^[5]
- "the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software";^[6]
- "an engineering discipline that is concerned with all aspects of software production";^[7]
- and "the establishment and use of sound engineering principles in order to economically obtain software that is reliable and works efficiently on real machines."^[8]

Operative Words

Typical formal definitions of **software engineering** are:

- "the systematic application of scientific and technological knowledge, methods, and experience to the design, implementation, testing, and documentation of software";^[5]
- "the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software";^[6]
- "an engineering discipline that is concerned with all aspects of software production";^[7]
- and "the establishment and use of sound engineering principles in order to economically obtain software that is reliable and works efficiently on real machines."^[8]

The Game of SE is Multi-Player with Releases

"[Software Engineering] is the **Multi-Person** construction of **Multi-Version** programs."

—- Dave Parnas* (ca 1974)

This implies teams with all sorts of socio issues as well as needing to iterate our code to understand SE

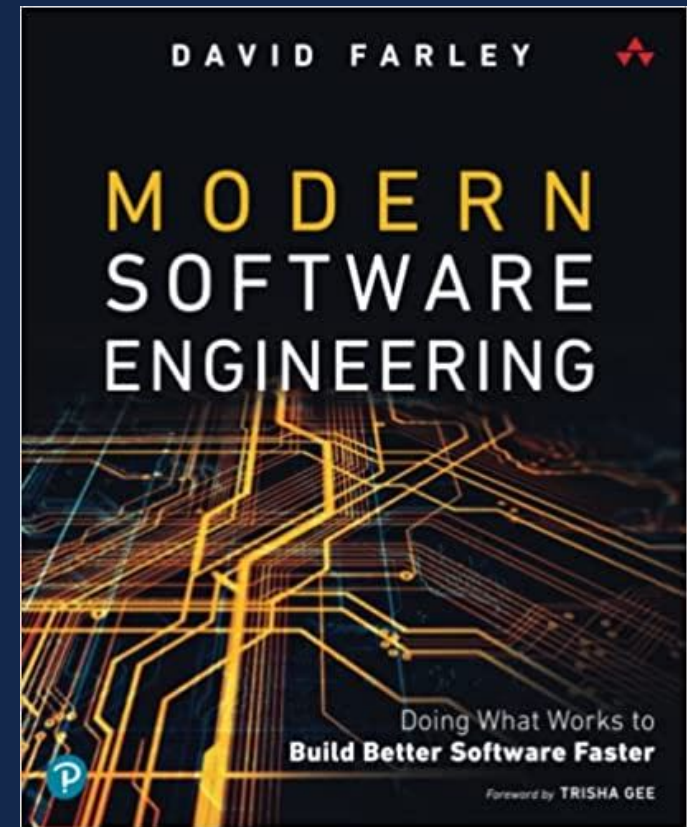
Teams introduces lots of challenges, but time does as well since with only 10 weeks we need to keep dev idea small or we simply can't do the second point

Time Range Clarification?

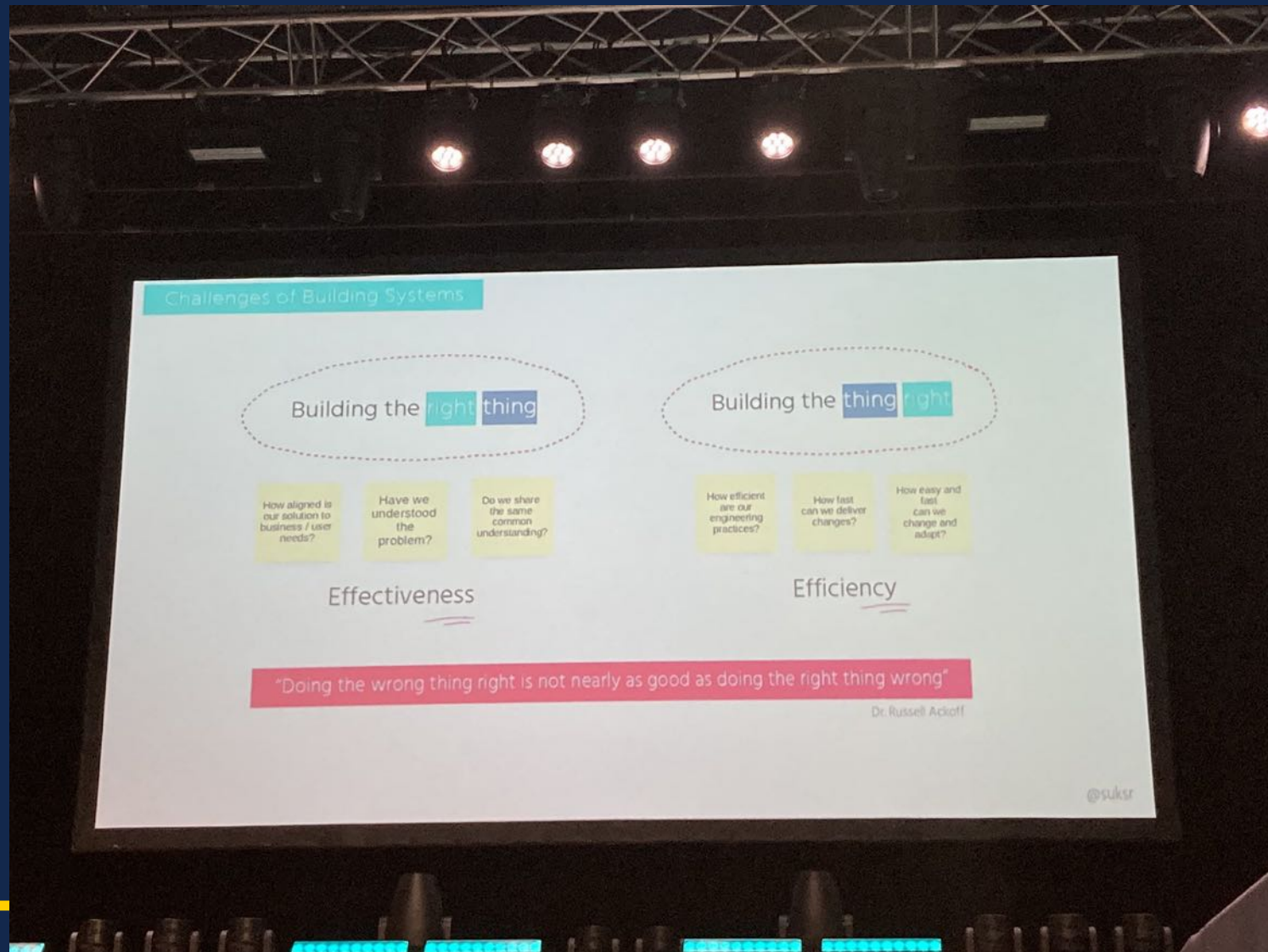


Farley on SWE

“Software engineering is the application of an empirical, scientific approach to finding efficient, economic solutions to practical problems in software.”



Design Engineering and Process Engineering



Ouch!



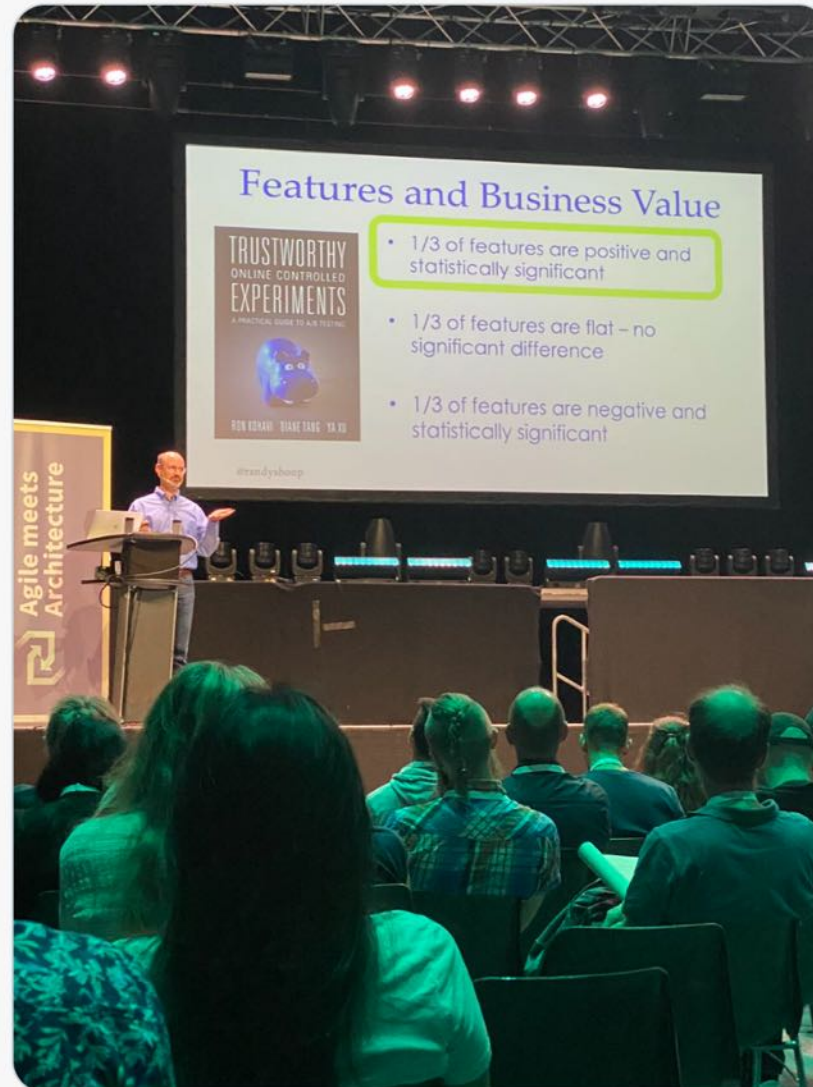
Joshua Kerievsky 🇺🇸 @JoshuaKerievsky · Sep 6

...

In his opening keynote, @randyshoup explains how Kohavi, Tang and Xu found that 2/3rds of features add no value.

“Building the wrong thing is the biggest waste in software” —Mary and Tom Poppendieck.

#amaberlin



Knowing what
to build
appears to be
the much
harder part



5



28



79



Less How, More What

In the past engineering has been dominated to a great extent by “what can we do,” but now “what do we want to do” looms greater since we now have the power to design almost anything we want.

The Art of Doing Science and Engine...

Richard W. Hamming and Bret Victor

Engineering
Learning
to Learn

Richard
Hamming

What is it and do we even understand?



Chris Gioran ❤️

@chrisg

1d

When I tell people outside my bubble that I am a programmer, they more often than not ask me what that is. Like, they know the word, but they don't know what I actually do.

I find that very odd.

Everyone knows what a doctor or a lawyer does, but they interact with them much less than with the product of a programmer's labor. They use a phone every minute of every day, for example.

But the best way I can explain what I do to a non programmer is "I work with computers".

7

5

7



Matthew Skelton

@matthewskelton@mastodon.social

@chrisg I explain programming as "encoding organisational intent"

1

1

Huh!?

Jun 9, 2024 at 2:25 AM

REPLIES



Chris Gioran ❤️

@chrisg

1d

@matthewskelton That's accurate (mostly - a simple web app is not that complicated), but my point is not about the difficulty of explaining what programming is.

It's about the unfamiliarity of the concept outside the circle of practitioners despite its ubiquity

2

2



Matthew Skelton

@matthewskelton@mastodon.social

1d

@chrisg I would argue that many programmers don't actually understand what they are really doing, otherwise there would be far more interest and care for DDD, TDD, sociotechnical aspects, etc.

11

2



Matthew Skelton

@matthewskelton@mastodon.social

1d

@chrisg too many folks are like "make 'puter go brrrrr" with little further thought.

1

1

Whoa buddy

Ok spicy quotes not from a rando



My less spicy and expanded version of what he is saying is that our job is to ...

Encoding organizational intent to satisfy end user needs or wants in the form of software systems which may include us writing code, using software/human systems, or some combination there of.

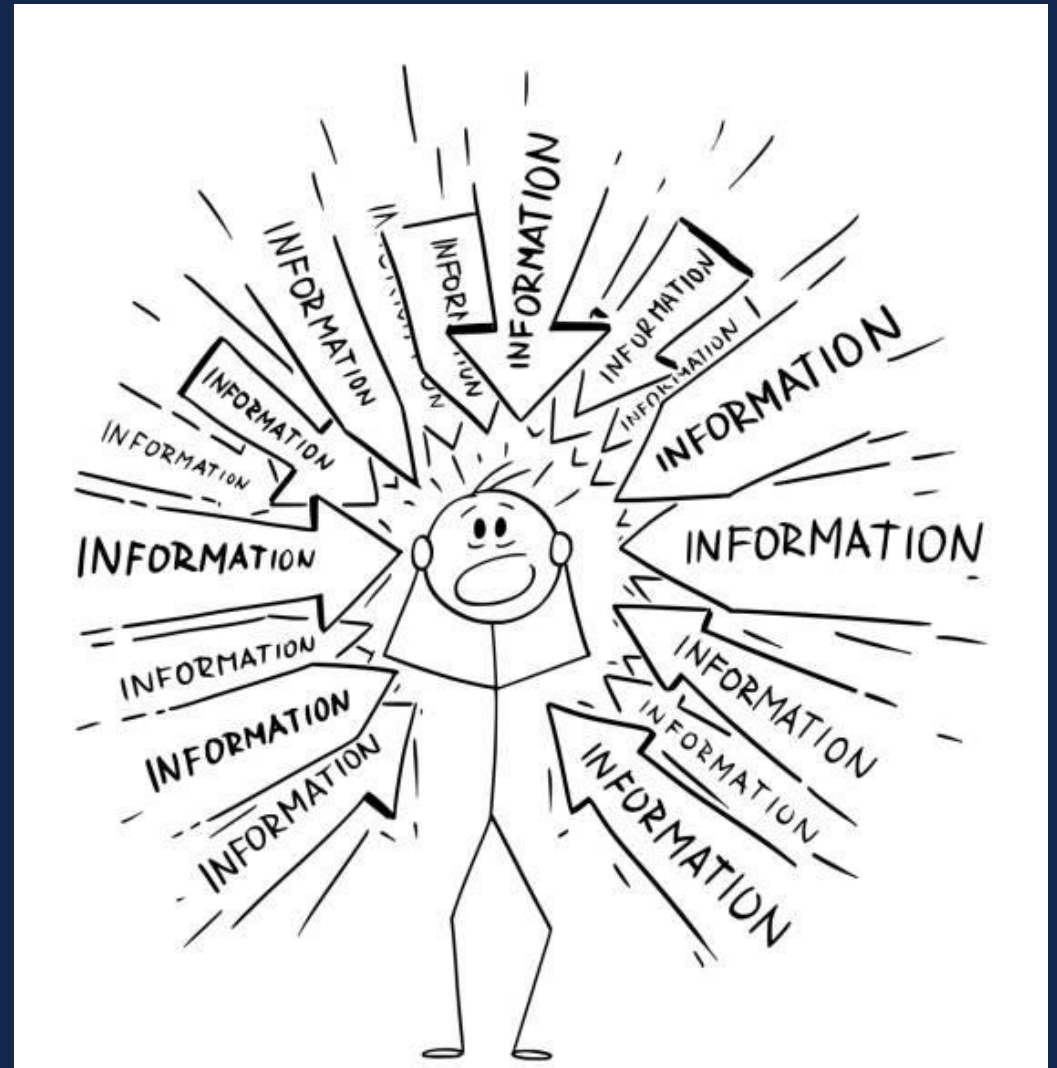
Regardless of definition
variability we can say 100%
for sure...it's huge!!!

SWEBOK (Software Engineering Body of Knowledge)

Table I.1. The 15 SWEBOK KAs

Software Requirements
Software Design
Software Construction
Software Testing
Software Maintenance
Software Configuration Management
Software Engineering Management
Software Engineering Process
Software Engineering Models and Methods
Software Quality
Software Engineering Professional Practice
Software Engineering Economics
Computing Foundations
Mathematical Foundations
Engineering Foundations

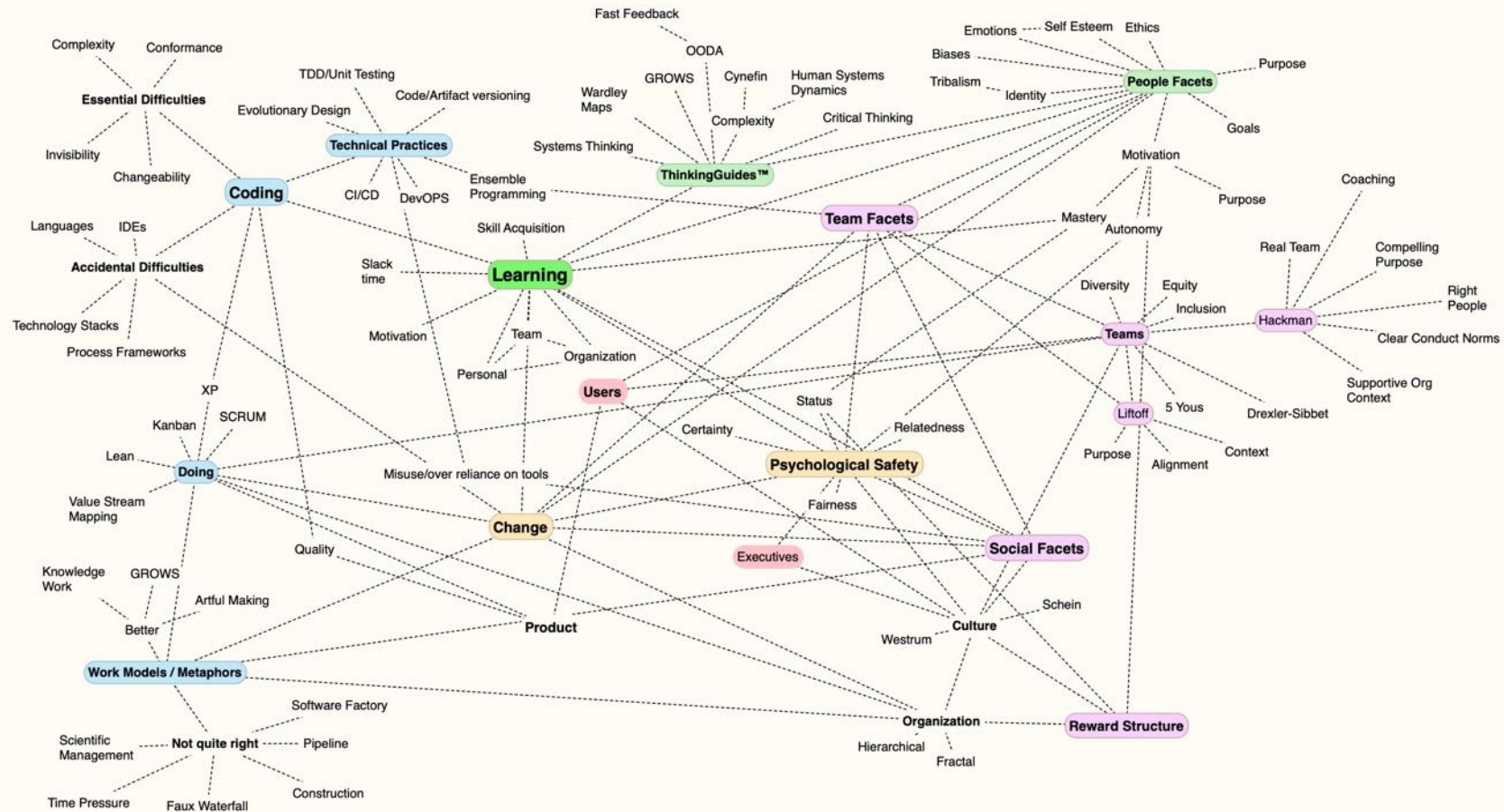
<http://www.swebok.org/>



Depiction of CSE Students Dealing with SE in 1 quarter

A Lot Less Tech In Here!?!?

A system is not the sum of its parts, but the product of its interactions. Ackoff

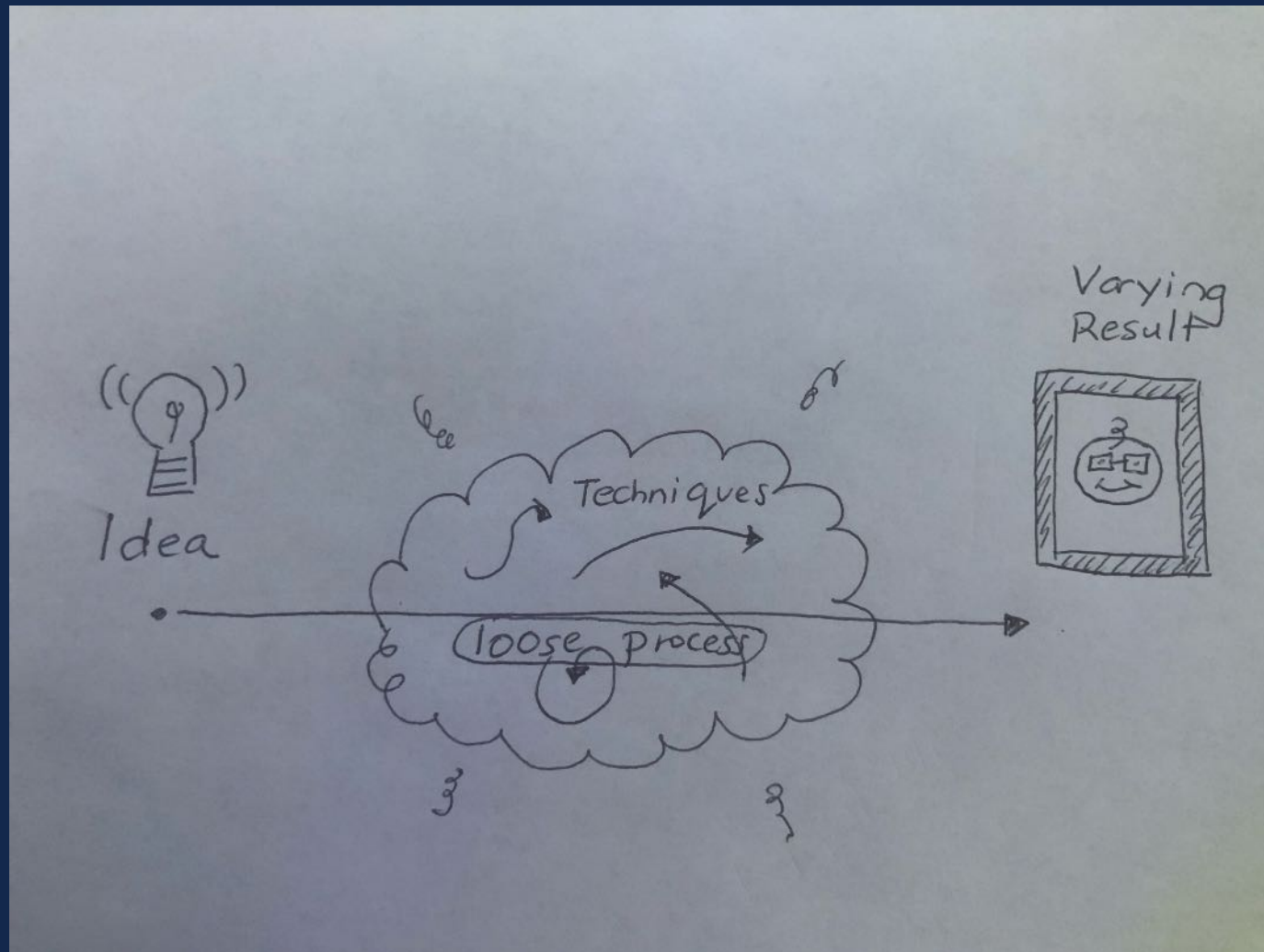
$$x(5)^5 * x(4)^4 * x(3)^3 * x(2)^2 * x(1) = \text{output}$$


This map is a work in progress, and always will be. Succeed at your own risk.

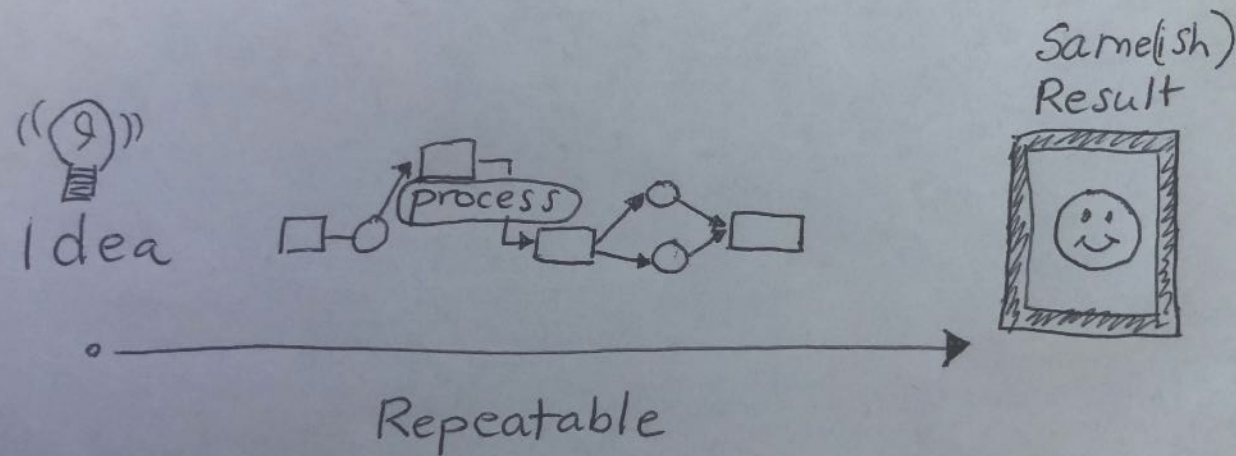
©2019 - 2022 Don Gray

Like many are saying are
we really doing it?

Artistry or Craft?



The Ultimate Idea of Engineering



So to do that what do we need to focus on?

The most important thing for a software engineer is...

Coding duh!

Whatever is trending on Reddit

Thinking

Organizing

Communicating

Algorithms and Data Structures

npm install!

Typing Speed

Memorizing Leetcode Answers

Grinding 996 style!

Prompt Engineering!

All of this and more!?!

Generally...

- Given how we likely learned and experienced things so far it would tend to favor the “coding” and “algo” parts of that last little exercise
 - But is that actually right? Hopefully, after our exercises you are skeptical that might not be right.
-

Nope...Software Engineering > Programming

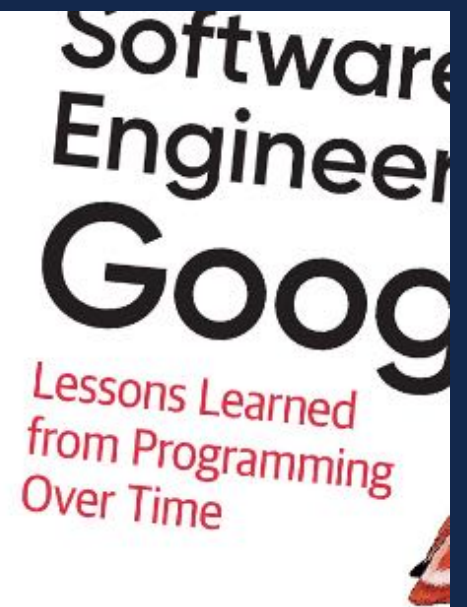
“Anyone who writes code is considered a programmer. Some of us who write code are not only programmers, but Software Engineers. As engineers we architect and design our systems in a rigorous and repeatable manner employing thinking and skills far beyond mere coding that consider the environment we code within and the people we do it for and with.”

Yet Again Not Just Me Thinking This

Programming is the immediate act of producing code. Software engineering is the set of policies, practices, and tools that are necessary to make that code useful for as long as it needs to be used and allowing collaboration across a team.

Software Engineering at Google

Titus Winters, Tom Manshreck, and H...



Say it again 1000s of times...SE > Code

We propose that “software engineering” encompasses not just the act of writing code, but all of the tools and processes an organization uses to build and maintain that code over time.

Software Engineering at Google

Titus Winters, Tom Manshreck, and H...

Software
Engineer
Goog
Lessons Learned
from Programming
Over Time



Activity sheet time!

Did we have a clear consistent definition?

That and our respect for this is a BIG problem for us and that's why I hope you want to improve. Let's see what's at stake

Meet a Cranky Old SE Man



Alan Cooper ✓

@MrAlanCooper

Follow



You know, I've been in the software development business for more than 40 years now, and there is one thing that I know for a solid fact: Nobody knows how to build software. Yes, sometimes it gets built, but it's a random, unrepeatable event. A lucky accident.

4:38 PM - 30 Mar 2019

1,106 Retweets 4,367 Likes



87



1.1K



4.4K

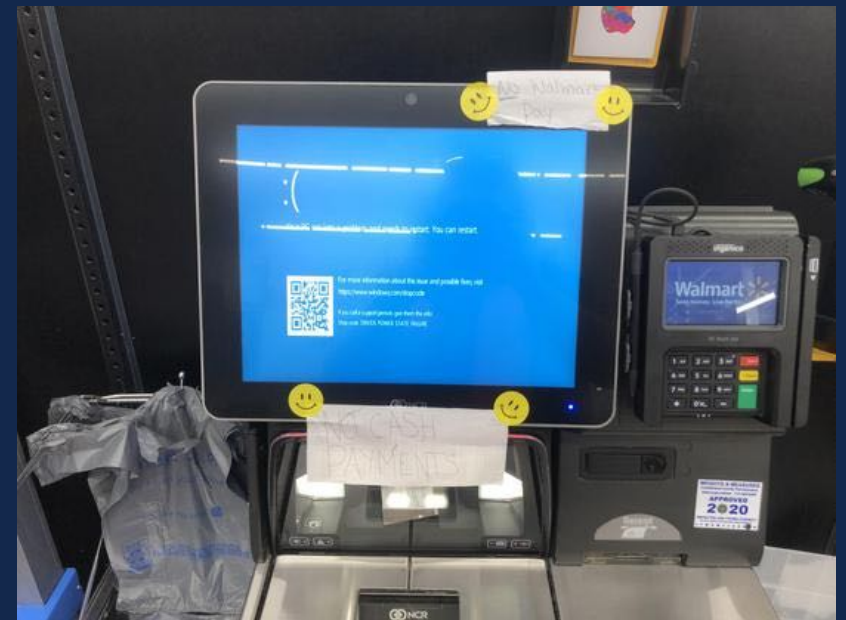


The Old Cranky Guy Had a Point

The aggregated impacts of software quality shortcomings across an organization, a nation, or the planet as a whole are simply staggering. A detailed analysis estimated **the total costs of poor software quality in the United States in 2018 at approximately \$2.26 *trillion* if technical debt is not included and \$2.84 trillion if it is (Krasner 2018)**. Just imagine the economic benefits — at every level — that higher-quality software could yield. One way to reduce the cost of poor quality is to correct defects early in the development cycle, before they reach the user.

<https://www.it-cisq.org/the-cost-of-poor-quality-software-in-the-us-a-2018-report/The-Cost-of-Poor-Quality-Software-in-the-US-2018-Report.pdf>

All This Deeply Matters 'Member This?



Ancient Times - 2024

Modern SWE in Industry?

Software engineers at the cybersecurity firm CrowdStrike complained about rushed deadlines, excessive workloads, and increasing technical problems to higher-ups for more than a year before a catastrophic failure of its software paralyzed airlines and knocked banking and other services offline for hours.

“Speed was the most important thing,” said Jeff Gardner, a senior user experience designer at CrowdStrike who said he was laid off in January 2023 after two years at the company. “Quality control was not really part of our process or our conversation.”

...but it's all good though



404 Media

@404mediaco@mastodon.social

Affected by CrowdStrike outage? Simply try rebooting 15 consecutive times, Microsoft suggests

404media.co/to-fix-crowdstrike...



404 Media · Jul 19, 2024

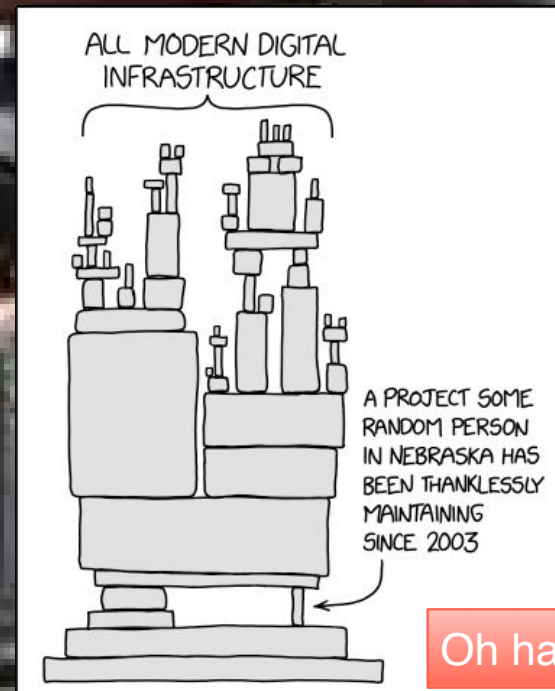
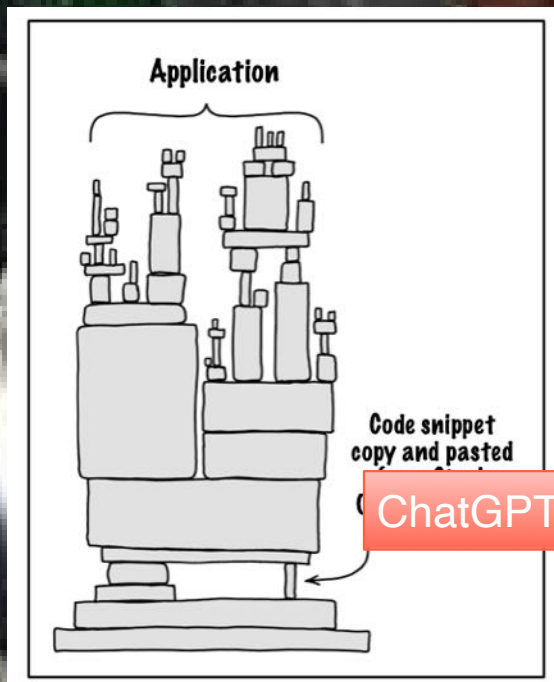
To Fix CrowdStrike Blue Screen of Death Simpl...

The advice, which is specifically for virtual machines using Azure,...



CrowdStrike PR Flack

TL;DR Modern SW Engineering Visualized



It gets worse...ominous foreshadowing...



Brandon Weaver @keystonelemur · Aug 11



The longer I work in tech the more I'm convinced that the hardest problems are more social than they are technical, and that pretending otherwise is what frequently makes for some of the most confoundedly hard systems to work with.

 38

 229

 1,436





Thomas A. Powell @thomasapowell · Apr 29, 2020



Can confirm. Tool syntax problems are the easy predictable part of software engineering and while they are challenging their value is time limited compared to deeper harder lessons in thinking and people. Work on those and excel long term



Ian Coldwater 📦💣🔵 @IanColdwater · Apr 29, 2020

junior engineer: I'm going to learn all the tools and syntax! when I'm senior, this knowledge will help me solve important problems

senior engineer: wtf? the most important problems are people problems, this doesn't have anything to do with code at all



I'm long gone from that place don't worry...but note the date!



Sarah Mei  @sarahmei · Mar 13, 2018

We're starting to differentiate "software developers" from "software engineers" and this absolutely nails the difference.

An engineer builds software systems that run in the context of people.

A developer writes code.



cron mom @sophaskins · Mar 12, 2018

There are no good engineers who only pay attention to code
There are no good engineers who only think about technologies
There are no good engineers who ignore social issues
The human context of your work is what makes engineers good or bad

[Show this thread](#)



49



371

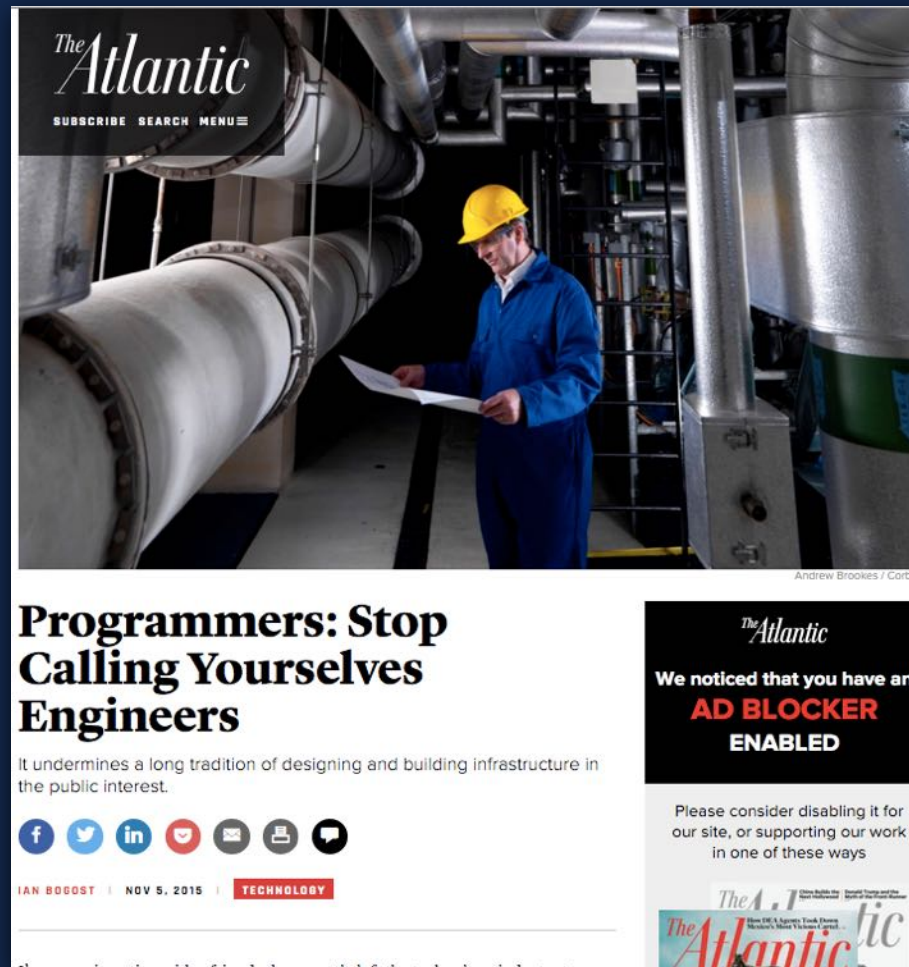


1K



Note the dates...this is not some "discovery"

A Helpful Assignment Coming?



<http://www.theatlantic.com/technology/archive/2015/11/programmers-should-not-call-themselves-engineers/414271/>

All To Help You Be Open to Checking Your Priors

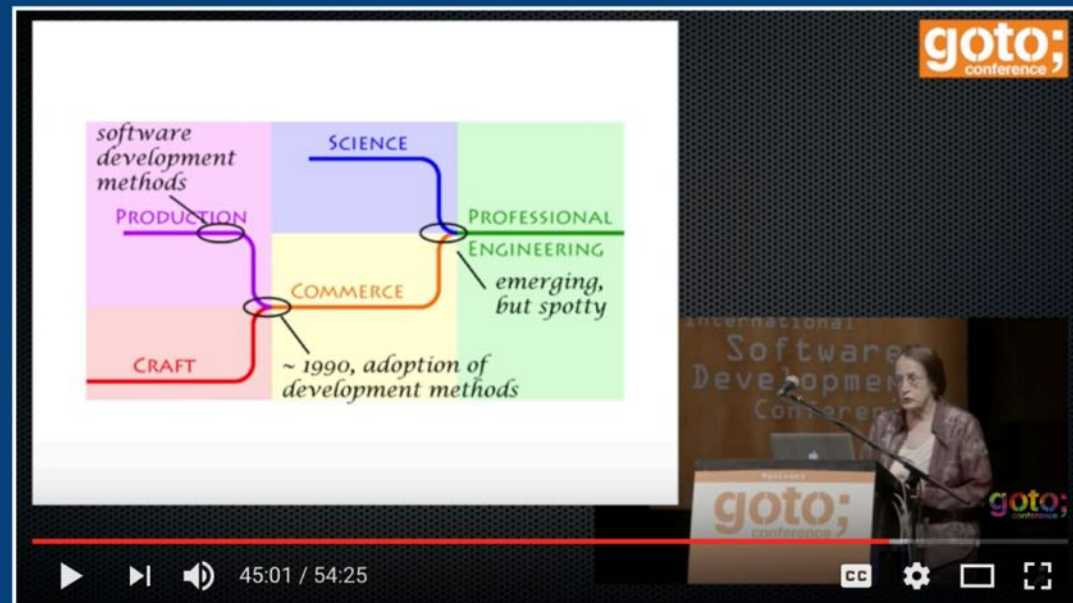
Part of the mistake here is to assume some level of idealistic precision that is impossible in any field, let alone the field of software development. We have made the mistake of seeking mathematical precision, which is not the same thing as engineering!

Modern Software Engineering

David Farley



And know this whole debate is long running...



Progress Toward an Engineering Discipline of Software

Mary Shaw

Logistics Review

- Sign-up for Slack
 - An individual assignment coming soon and survey coming soon
 - But first...
-

Oh wait...all this is gonna
change because AI!?!

Activity: Fill in the last three questions on your sheet

My Take (Last Spring)

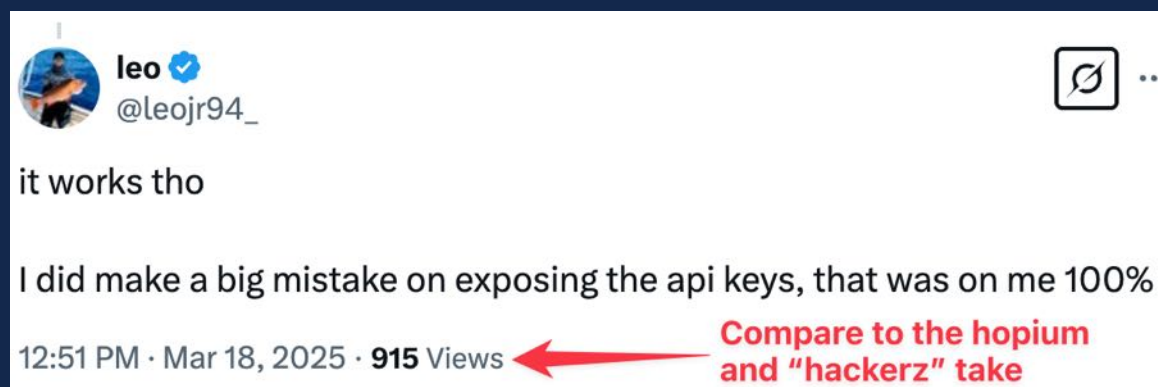
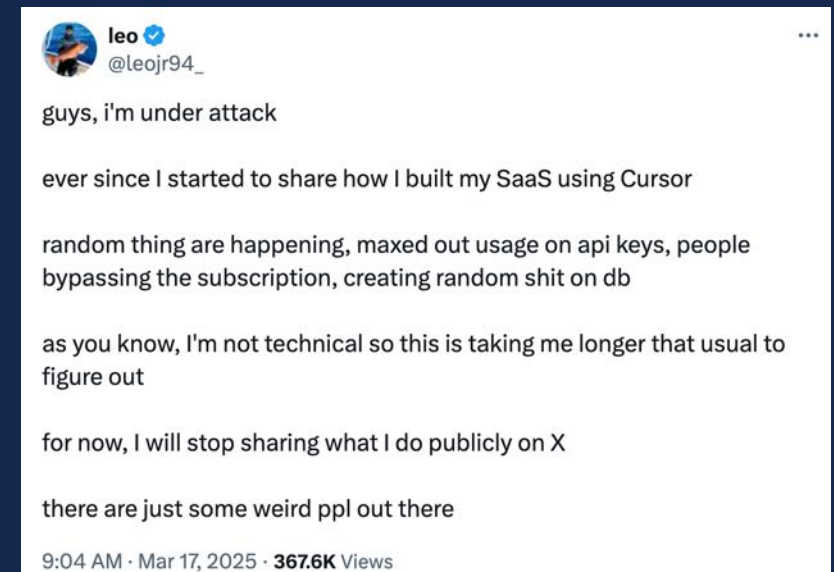
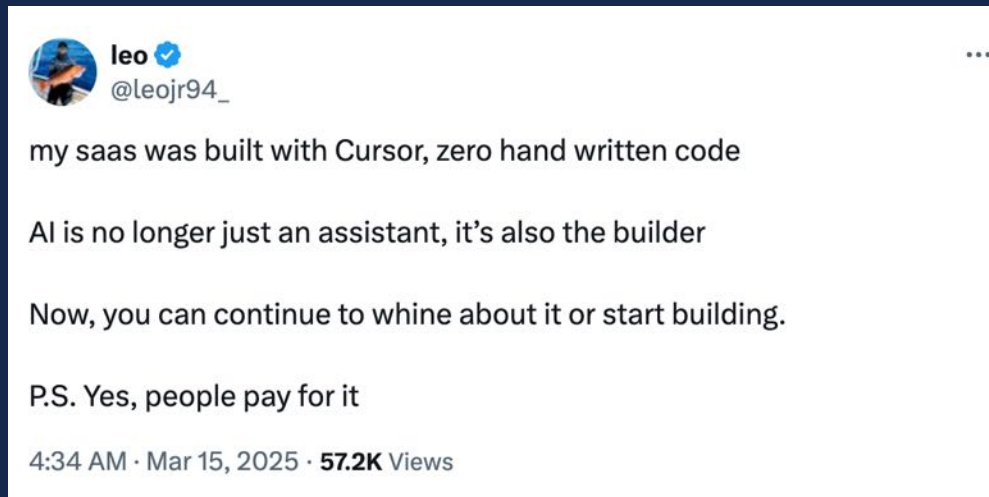
- Take hype at face value that it could work then it means you need to do the non-code stuff a lot more like getting requirements, be better at precise writing of English specifications (aka prompts), and more “design engineering” as code generation will be less useful
 - Assuming it isn't that great, it means you likely have to still deeply learn the fundamentals so you can verify what is produced. Also you would need to make an honest assessment of use risk in terms of quality, legal issues, etc. and mitigate that including not using it.
 - '25 - Introducing...
-

VIBE CODING



Errr...how do you see
yer code on this laptop?

A Vibe Coding Adventure in Three Acts



Compare to the hopium
and "hackerz" take

Kind of Obvious Why

Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions

Hammond Pearce Baleegh Ahmad Benjamin Tan Brendan Dolan-Gavitt Ramesh Karri
Department of ECE Department of ECE Department of ESE Department of CSE Department of ECE
New York University New York University University of Calgary New York University New York University
Brooklyn, NY, USA Brooklyn, NY, USA Calgary, Alberta, CA Brooklyn, NY, USA Brooklyn, NY, USA
hammond.pearce@nyu.edu bal283@nyu.edu benjamin.tan1@ucalgary.ca brendandg@nyu.edu rkarri@nyu.edu

we produce 89 different scenarios for Copilot to complete, producing 1,689 programs. Of these, we found approximately **40 % to be vulnerable**.

To perform this analysis we prompt Copilot to generate code in scenarios relevant to high-risk cybersecurity weaknesses, e.g. those from MITRE’s “Top 25” Common Weakness Enumeration (CWE) list. We then analyze the code generated by Copilot to see if it contains any of the weaknesses. In total, we produced 1,689 programs, of which 40 % to be vulnerable. Index Terms: AI assistants, code generation, C

Do Users Write More Insecure Code with AI Assistants?

Neil Perry* Megha Srivastava* Deepak Kumar Dan Boneh
Stanford University Stanford University Stanford University / UC San Diego Stanford University

ABSTRACT risks of AI assistants in the context of how developers choose to

we find that participants who had access to an AI assistant wrote significantly less secure code than those without access to an assistant. Participants with access to an AI assistant were also more likely to believe they wrote secure code, suggesting that such tools may lead users to be **overconfident** about security flaws in their code.

• Security and privacy → Human and societal aspects of security and privacy; Participants with access to an AI assistant wrote insecure solutions more often than those without access to an AI assistant for

arXiv > cs > arXiv:2503.11082

Search... Help | Advan...

Computer Science > Software Engineering

[Submitted on 14 Mar 2025]

LLMs are Bug Replicators: An Empirical Study on LLMs' Capability in Completing Bug-prone Code

Liwei Guo, Sixiang Ye, Zeyu Sun, Xiang Chen, Yuxia Zhang, Bo Wang, Jie M. Zhang, Zheng Li, Yong Liu

Large Language Models (LLMs) have demonstrated remarkable performance in code completion. However, the training data used to develop these models often contain a significant amount of buggy code. Yet, it remains unclear to what extent these buggy

in bug-prone tasks, the **likelihood of LLMs generating correct code is nearly the same as generating buggy code**, and it is substantially lower than in normal code completion tasks. To our surprise, **44.44% of the bugs LLMs make are completely identical to the pre-fix version**, indicating that LLMs have been seriously biased by historical bugs

Next Up

- Individual SE skills and goals
 - Individual goals assignment
 - Group survey
-